

Zip Code Group Project 2.0 — Design Document

Team 5

St Cloud State University

CSCI 331 - Software Systems — Spring 2025

Dr. Anda, Andrew

March 22nd, 2025

*Written with the utmost dedication and **last-minute** inspiration.*

Table of Contents

Table of Contents.....	2
Overview.....	3
What is LRF?.....	3
Features and Functionality.....	3
Data Structures.....	5
1. CSVBuffer.....	5
2. LRFBuffe.....	5
3. HeaderRecordBuffer.....	5
4. StateExtremesReport.....	6
How It Works:.....	6
Example Output:.....	7
5. Primary Key Index.....	8
Functionality Breakdown.....	8
1. printCSVRecords(CSVBuffer& buffer).....	8
2. printLRFRecords(LRFBuffe.....	8
3. readHeaderRecord(const std::string& filename).....	9
4. writeHeaderRecord(const std::string& filename).....	9
5. searchZipCodeRecords(const std::string& lrfFilename).....	9
User Interface.....	10
Error Handling.....	10
Additional Considerations.....	11
Future Improvements.....	11

Overview

This program provides functionality for reading, printing, updating, and converting data between CSV and LRF formats. The program allows the user to interact with these data formats through a simple console-based menu system. Users can perform various operations, such as viewing records from CSV and LRF files, converting CSV to LRF, reading and writing header records, and searching for specific data using an index.

What is LRF?

LRF (Length-Related Format) is our custom file format designed for length-indicated records, optimized for efficient storage and retrieval of structured data. Each record within an LRF file is prefixed by a length indicator that specifies the size of the record, enabling fast and precise parsing of variable-length data. This format is ideal for applications that need to handle large datasets with varying record sizes, allowing for efficient indexing and searching, particularly when dealing with records such as zip codes or other primary key fields. The LRF format streamlines data management, ensuring that both sequential reading and random access operations remain performant and reliable.

Features and Functionality

The program offers the following functionalities:

1. **Print CSV Records:** Displays the records from a CSV file in a readable format.
2. **Print LRF Records:** Displays the records from a LRF file in a readable format.
3. **Convert CSV to LRF:** Converts data from a CSV file into the LRF format.
4. **Read Header Record:** Reads and displays the header information from an LRF file.
5. **Write (Update) Header Record:** Allows the user to modify and save changes to the header record of an LRF file.

6. **Run State Extremes Report:** Generates a state extremes report based on LRF data (functionality details for this are not provided in the code).
7. **Search for Zip Code Records by Index:** Searches for records by Zip Code using an index file and displays matching records.

Data Structures

1. CSVBuffer

This class handles the reading and parsing of CSV files.

- **Key Methods:**
 - `getHeaders()`: Returns a list of column headers in the CSV file.
 - `getNextRecord()`: Retrieves the next record in the CSV file.

2. LRFBuffer

This class handles the reading of LRF files.

- **Key Methods:**
 - `getHeaders()`: Returns a list of headers for the LRF file.
 - `getNextRecord()`: Retrieves the next record in the LRF file.

- `seekTo()`: Allows random access to specific positions in the LRF file using its accompanying index file.

3. HeaderRecordBuffer

This class is responsible for reading and writing header data in an LRF file.

- **Key Methods:**

- `readHeader()`: Reads the header record from the file.
- `writeHeader()`: Writes the modified header back to the file.
- `getFileType()`, `getVersion()`, `getRecordLengthFieldSize()`, etc.: Methods to retrieve specific header information.
- `setFileType()`, `setVersion()`: Methods for updating header data.

4. StateExtremesReport

The `StateExtremesReport` is a program designed to generate a report detailing the geographic extremes (easternmost, westernmost, northernmost, and southernmost) for each state based on zip code data. It uses a custom `LRFBuffer` to read and process data from an LRF file, which contains records of zip codes along with their geographic coordinates (latitude and longitude).

How It Works:

1. **StateExtremes Structure:** The program uses the `StateExtremes` structure to store the zip codes and their corresponding geographic extremes for each state. This structure tracks the four extremes for each state: the easternmost, westernmost, northernmost,

and southernmost zip codes, as well as their respective latitude and longitude values.

2. Data Processing:

- The program iterates over each record in the LRF file, extracting the zip code, state, latitude, and longitude.
- For each record, the program checks if the state is already present in the `stateMap`. If not, it initializes a new entry with the first record's data.
- If the state already exists in the map, the program compares the current record's latitude and longitude to update the extremes for that state.

3. Output:

- Once all records are processed, the program outputs a table displaying the extremes for each state, including the zip codes associated with each extreme, neatly formatted in columns.

4. **Error Handling:** The program wraps the entire process in a try-catch block to catch and display any errors that might occur during file processing or data manipulation.

Example Output:

```
State | Easternmost Zip | Westernmost Zip | Northernmost Zip | Southernmost Zip
-----
CA    | 90210           | 96013           | 94920           | 91905
TX    | 77571           | 79901           | 33.1            | 25.5
...
```

This report helps to quickly identify the geographic extremes for each state based on zip code data, which can be valuable for various applications such as geographic analysis or distribution planning.

5. Primary Key Index

The index file (.idx) is used to quickly locate records in the LRF file by their Zip Code.

- **Key Operations:**
 - The index is stored as a mapping of Zip Code to file position (offset).
 - The program uses this index to seek specific records in the LRF file efficiently.

Functionality Breakdown

1. printCSVRecords(CSVBuffer& buffer)

- Displays the column headers of the CSV file followed by each record's data.
- **Input:** A `CSVBuffer` object containing the parsed CSV file.
- **Output:** Printed output of the records and headers.

2. printLRFRecords(LRFBuffer& buffer)

- Similar to `printCSVRecords`, but for LRF files.
- **Input:** A `LRFBuffer` object containing the parsed LRF file.
- **Output:** Printed output of LRF records.

...

3. readHeaderRecord(const std::string& filename)

- Reads the header record of an LRF file and displays its metadata, such as file type, version, record count, and field definitions.
- **Input:** The file name of the LRF file.
- **Output:** Printed header information.

4. writeHeaderRecord(const std::string& filename)

- Prompts the user to update specific fields (File Type, Version) of the header record in an LRF file and saves the changes.
- **Input:** The file name of the LRF file, followed by user input for new header values.
- **Output:** Success message upon updating the header.

5. searchZipCodeRecords(const std::string& lrfFilename)

- Allows users to search for specific Zip Code records by using an index file.
- **Input:** A list of Zip Code flags (e.g., "-Z56301").
- **Output:** The matching record for each Zip Code (if found), displayed from the LRF file.

User Interface

The user interacts with the program through a text-based menu system:

1. Print CSV Records
2. Print LRF Records
3. Convert CSV to LRF
4. Read Header Record
5. Write (Update) Header Record
6. Run State Extremes Report
7. Search for Zip Code Records by Index

The program prompts the user to enter a choice and provides input prompts for each selected operation. For file handling (e.g., CSV or LRF files), the user is prompted for file names. If a user selects a search or update operation, they are guided through entering the relevant information (such as new header values or Zip Codes).

Error Handling

- **File I/O errors:** If the program cannot open a file (e.g., CSV, LRF, or index file), it displays an error message and terminates or prompts the user to retry.
- **Invalid User Input:** If the user provides invalid data (e.g., incorrectly formatted Zip Codes), the program will notify the user and proceed without crashing.
- **Exceptions:** The program is wrapped in a try-catch block to catch and report errors during execution, preventing crashes.

Additional Considerations

- **Modularity:** The program relies on separate classes and functions for different operations (CSV handling, LRF handling, header management), making the code modular and maintainable.

Future Improvements

- **Error Handling Enhancements:** More robust checks for file integrity and better error messages for user guidance.
- **StateExtremesReport Implementation:** The state extremes report feature could be further detailed to process and analyze LRF data for statistical analysis or other forms of reporting.
- **Looping Menu:** Currently after the user completes an action they must rerun the program to perform the next action. Having the menu loop until the user exits could be useful for workflow if implemented.
- **Data Validation:** The program could be extended to validate data formats (e.g., ensuring that Zip Codes are valid) before performing actions like searching.